



# Projekt: Gesten-Spiele für den Tag der offenen Tür

Arbeiten im Team mit KI · Agent in VS Code · Chat im Browser

## Worum es geht

Am Samstag, 10.10., ist Tag der offenen Tür. Besucher sollen dort Spiele spielen, ohne Tastatur und Maus. Eine KI erkennt über die Kamera Hände und Körper. Die Spiele bauen Sie in den nächsten zwei Wochen, und Sie setzen dabei selbst KI ein, um den Code zu schreiben.

Die Zeit ist knapp. Kamera, Erkennung und Server sind deshalb schon fertig, Sie bauen nur das Spiel. Jedes Zweierteam arbeitet an einer eigenen Datei im Ordner `module/`. Wie die Dateien zusammenpassen, steht in `SCHNITTSTELLE.md`.

| Termin    | Was passiert                                                                                                           |
|-----------|------------------------------------------------------------------------------------------------------------------------|
| Di 29.09. | Start: Repo anlegen und klonen, Teams, erstes Ticket                                                                   |
| Fr 02.10. | Ticket 1 fertig, Ticket 2 läuft                                                                                        |
| Di 06.10. | Probelauf mit Leuten, die das Spiel nicht kennen. Danach <b>Feature-Freeze</b> : nur noch Fehler beheben, nichts Neues |
| Fr 09.10. | Generalprobe am Aufbauort                                                                                              |
| Sa 10.10. | Tag der offenen Tür                                                                                                    |

## Ihr Team

Team und Spiel: \_\_\_\_\_

Ihre Datei: \_\_\_\_\_

Ihre Tickets stehen in: \_\_\_\_\_

In einem Ticket steht, was Ihre Datei danach können soll, und woran man erkennt, dass es fertig ist (Akzeptanzkriterien). Ein Ticket ist erledigt, wenn alle Kriterien vor der Kamera oder mit der Maus erfüllt sind.

**Die wichtigste Regel im Projekt:** Sie ändern nur Ihre eigene Datei. Alle arbeiten gleichzeitig auf `main`. Ändern zwei Teams dieselbe Datei, gibt es Konflikte, und ein Fehler bei Ihnen legt das Spiel der anderen lahm.

## Rollen im Team

| Rolle    | Aufgabe                                                                                                                  |
|----------|--------------------------------------------------------------------------------------------------------------------------|
| Pilot    | arbeitet mit der KI: schreibt die Anfrage, übernimmt oder verwirft das Ergebnis                                          |
| Co-Pilot | liest jede Anfrage, bevor sie abgeschickt wird. Testet, hakt die Akzeptanzkriterien ab, macht Commit und Synchronisieren |

Die Rollen wechseln jede Stunde. Beide arbeiten also mit der KI, und beide prüfen.

## Zwei Wege, mit der KI zu arbeiten

|                         | Weg A: Agent in VS Code                                                                                    | Weg B: Chat im Browser                                                               |
|-------------------------|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Werkzeug                | GitHub Copilot im Modus <a href="#">Agent</a> ,<br>oder eine Agent-App wie ChatGPT<br>im Agentenmodus      | Microsoft Copilot, ChatGPT oder<br>eine andere KI Ihrer Wahl                         |
| Was die KI sieht        | das ganze Projekt. Sie liest Dateien<br>selbst und ändert Ihre Datei direkt                                | nur das, was Sie hineinkopieren                                                      |
| Wie der Code ankommt    | VS Code zeigt die Änderungen, Sie<br>behalten oder verwerfen sie                                           | Sie kopieren die Antwort und<br>ersetzen Ihre Datei                                  |
| Kontingent              | bei GitHub Copilot Free etwa 200<br>Anfragen im Monat je Konto, bei<br>anderen Apps je nach Anbieter       | je nach KI, meist ein Tageslimit                                                     |
| Die Regeln fürs Projekt | stehen in <code>copilot-instructions.md</code><br>bzw. <code>AGENTS.md</code> und gehen<br>automatisch mit | stehen in <code>prompt-vorlagen/</code> , Sie<br>fügen sie am Anfang jedes Chats ein |

Beide Wege sind erlaubt. Ihr Team kann auch wechseln, zum Beispiel wenn das Kontingent im Agenten aufgebraucht ist. Der Agent spart das Kopieren und sieht mehr. Dafür sind seine Anfragen begrenzt, und jede soll sitzen.

## Weg A · GitHub Copilot einrichten

Das macht jede Person einmal, am eigenen Laptop. Sie brauchen Ihr GitHub-Konto aus dem Git-Crashkurs. GitHub Copilot ist in VS Code schon eingebaut und in einer kostenlosen Stufe (Copilot Free) nutzbar.

1. Öffnen Sie in VS Code den Projektordner. Fragt VS Code, ob Sie den Autoren der Dateien vertrauen, klicken Sie auf [Ja, ich vertraue den Autoren](#) .
2. Öffnen Sie den Chat mit Strg+Alt+I. Er geht rechts als Seitenleiste auf.
3. Klicken Sie im Chat auf [KI-Features mit Copilot kostenlos verwenden...](#) und dann auf [Weiter mit GitHub](#) .
4. Im Browser melden Sie sich bei GitHub an und erlauben VS Code den Zugriff. Fragt der Browser, ob er Visual Studio Code öffnen darf, bestätigen Sie. Copilot Free wird dabei für Ihr Konto eingeschaltet.
5. Zurück in VS Code steht im Chat „Chat wird vorbereitet...“. Danach ist das Eingabefeld frei.

**Geklappt, wenn:** Unten rechts in der Statusleiste steht das Copilot-Symbol. Ein Klick darauf zeigt [Chatnachrichten](#) mit einer Prozentzahl. Das ist Ihr verbrauchtes Kontingent für diesen Monat. Schreiben Sie sie jetzt auf: \_\_\_\_\_.

### Stolperstellen beim Einrichten

„KI-Features werden derzeit nur in vertrauenswürdigen Arbeitsbereichen unterstützt.“ Sie haben in Schritt 1 abgelehnt. Schließen Sie den Ordner und öffnen Sie ihn neu.

**VS Code kommt nach der Anmeldung nicht wieder.** Wechseln Sie mit Alt+Tab zurück. Die Anmeldung ist erledigt, klicken Sie den Knopf noch einmal.

**Es geht kein Browser auf, oder GitHub lehnt ab.** Melden Sie sich im Browser zuerst direkt auf [github.com](https://github.com) an und versuchen Sie es dann erneut.

## Weg A · Die Regeln für den Agenten

Im Projekt liegt die Datei `.github/copilot-instructions.md`. VS Code schickt ihren Inhalt bei jeder Anfrage an den Agenten mit. Sie müssen die Regeln also nicht jedes Mal selbst schreiben. Sie stehen schon im Repo, wenn Sie es klonen. Agent-Apps wie ChatGPT lesen dieselben Regeln aus `AGENTS.md`. So sehen sie aus:

```
.github/copilot-instructions.md
```

### # Regeln für dieses Projekt

Ein Browser-Spiel für den Tag der offenen Tür. Mehrere Teams arbeiten gleichzeitig daran, jedes an einer eigenen Datei in `module/`.

- Ändere nur die Datei, die im Ticket genannt ist. Müsste eine andere Datei geändert werden, schreib das in die Antwort und ändere sie nicht.
- Halte dich an `SCHNITTSTELLE.md`. Benenne keine Methode und kein Feld um, das dort steht.
- Ändere nichts in `lib/` und `vendor/`.
- Reines JavaScript im Browser. Keine Bibliotheken, kein npm.
- Bezeichner auf Englisch, Texte auf dem Bildschirm auf Deutsch.
- Zahlen, die man beim Testen anpasst (Tempo, Größe, Zeiten), gehören in den `CONFIG`-Block am Anfang der Datei.
- Die Datei bleibt unter 250 Zeilen.
- Starte keinen Server und installiere nichts. Getestet wird von Hand im Browser.
- Erkläre zum Schluss in höchstens fünf Sätzen auf Deutsch, was du geändert hast.

Ändern Sie diese Datei und `AGENTS.md` nicht. Sie gelten für alle Teams. Fehlt eine Regel, sagen Sie es mir.

### Aufgabe 1 — Wozu die Regeln da sind

Suchen Sie sich zwei Regeln aus. Schreiben Sie je Regel auf, was ohne sie in diesem Projekt schiefgehen könnte.

---

---

---

---

## Weg A · Ein Ticket mit dem Agenten umsetzen

1. Synchronisieren, damit Sie den neuesten Stand der anderen Teams haben.
2. Im Chat oben auf `Neuer Chat`. Jedes Ticket bekommt einen eigenen Chat. Ein langer alter Chat macht den Agenten langsamer und ungenauer.
3. Unter dem Eingabefeld steht die Modusauswahl. Stellen Sie `Agent` ein.
4. Schreiben Sie die Anfrage. Nennen Sie Ticket und Datei. Der Co-Pilot liest mit, bevor Sie abschicken.

Setze T2 aus tickets/blasen.md um. Ändere nur module/blasen.js.

5. Der Agent liest Dateien und schreibt dann in `module/blasen.js`. Will er einen Befehl im Terminal ausführen, fragt er vorher. Klicken Sie nur auf [Zulassen](#), wenn Sie verstehen, was der Befehl tut. Laut den Regeln braucht er keinen.
6. Ist er fertig, zeigt VS Code die geänderten Zeilen grün und rot. Sehen Sie nach, welche Dateien er geändert hat. Steht dort eine andere als Ihre, klicken Sie bei dieser Datei auf [Rückgängig machen](#).
7. Laden Sie die Seite im Browser neu (F5) und testen Sie, zuerst mit der Maus (Taste M), dann mit der Kamera. Oben in Ihrer Ticketdatei steht die Adresse, unter der Ihr Teil allein läuft. Der Co-Pilot hakt die Akzeptanzkriterien ab.
8. Läuft es: [Beibehalten](#), dann Commit mit Ticketnummer, zum Beispiel `T2: Punkte und Zeitlimit`, und Synchronisieren. Läuft es nicht, siehe nächster Kasten.

#### Wenn es nicht läuft

Öffnen Sie im Browser die Konsole mit F12. Steht dort eine rote Fehlermeldung, kopieren Sie sie in den Chat, zusammen mit einem Satz, was Sie getan haben und was passiert ist. Ohne Fehlermeldung rät die KI.

Ist die Datei danach schlechter als vorher: [Rückgängig machen](#) verwirft alles, was der Agent in diesem Chat geändert hat. Haben Sie schon behalten, hilft in der Quellcodeverwaltung [Änderungen verwerfen](#). Damit sind Sie wieder beim letzten Commit.

#### Sparsam arbeiten: jede Nachricht zählt

Mit Copilot Free haben Sie **etwa 200 Anfragen im Monat** (Stand September 2026). Jede Nachricht, die Sie abschicken, verbraucht eine. Das gilt auch für „mach weiter“, „geht nicht“ und „danke“. Was der Agent danach an Dateien liest und schreibt, zählt nicht extra. Ist das Kontingent weg, meldet VS Code „Sie haben Ihr monatliches Kontingent an Chatnachrichten erreicht.“ Dann geht es erst im nächsten Monat weiter, das Projekt ist bis dahin vorbei.

Ihr Team hat zwei Konten, also etwa 400 Anfragen für drei Tickets. Das klingt viel. Ein Agent, der sich verrannt hat, verbraucht aber in einer Stunde schnell 30 davon. Und das Guthaben gilt für alles, was Sie in diesem Monat sonst mit Copilot machen.

## Gute Anfragen, schlechte Anfragen

| Kostet viele Anfragen                                | Kostet eine                                                                                                                                                                        |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| „Mach das Spiel besser.“                             | „Setze T3 aus <code>tickets/blasen.md</code> um. Ändere nur <code>module/blasen.js</code> .“                                                                                       |
| „Geht nicht.“                                        | „Nach dem Start bleibt der Bildschirm schwarz. In der Konsole steht: <code>TypeError: Cannot read properties of undefined (reading 'x')</code> , <code>blasen.js</code> Zeile 48.“ |
| „Die Blasen sind zu schnell.“                        | Selbst im CONFIG-Block die Zahl ändern. Dafür braucht es keine KI.                                                                                                                 |
| Drei Anfragen hintereinander für drei kleine Wünsche | Die drei Wünsche in einer Anfrage, als Liste                                                                                                                                       |

Bevor Sie abschicken, prüft der Co-Pilot drei Dinge: Steht die Ticketnummer drin? Steht die Datei drin? Ist bei einem Fehler die Meldung aus der Konsole dabei?

Zählen Sie mit. Hinter jedes Ticket kommt die Zahl der Anfragen (siehe KI-Notiz unten). Wer am Ende der ersten Woche schon mehr als die Hälfte seines Kontingents verbraucht hat, sagt im Stand-up Bescheid.

## Weg B · Mit einer KI im Browser

Hier arbeiten Sie mit einem Chat im Browser: Microsoft Copilot mit Ihrem Schulkonto, ChatGPT oder eine andere KI. Der Chat sieht Ihr Projekt nicht. Er weiß nur, was Sie ihm geben. Deshalb liegt für jedes Team im Ordner `prompt-vorlagen/` eine Datei mit Ihrem Teamnamen. Sie enthält dieselben Regeln wie `copilot-instructions.md` und dazu die Schnittstelle Ihres Moduls.

1. Synchronisieren.
2. Neuen Chat öffnen. Den ganzen Inhalt Ihrer Datei aus `prompt-vorlagen/` einfügen.
3. In dieselbe Nachricht darunter: den ganzen Inhalt Ihrer Datei aus `module/` und den Text des Tickets. Abschicken.
4. Die KI gibt die ganze Datei zurück. Markieren Sie in VS Code alles in Ihrer Datei (Strg+A), fügen Sie die Antwort ein und speichern Sie.
5. Testen wie bei Weg A. Läuft es: Commit und Synchronisieren. Läuft es nicht: Fehlermeldung aus der Konsole in denselben Chat.

### Stolperstellen bei Weg B

**Die KI gibt nur einen Ausschnitt zurück** („... der Rest bleibt gleich“). Schreiben Sie: „Gib die ganze Datei zurück.“ Stückweise einfügen geht fast immer schief.

**Die Antwort bricht mitten im Code ab.** Die Antwort war zu lang. Schreiben Sie „weiter“ und setzen Sie die beiden Teile zusammen, oder fangen Sie mit einem neuen Chat an.

**Die KI benennt Methoden um oder baut eine neue Datei.** Die Prompt-Vorlage fehlte, oder der Chat war schon lang. Neuer Chat, Vorlage zuerst.

**In den Chat gehört nur Quellcode.** Keine Namen, keine Passwörter, keine Kamerabilder. Das gilt für jede KI, egal bei welchem Anbieter.

## Git im Projekt

Alle arbeiten auf `main`, so wie in den vier Runden im Crashkurs. Branches und Pull Requests brauchen Sie hier nicht: Jedes Team hat seine eigene Datei, also kommen Sie sich beim Synchronisieren kaum in die Quere.

| Wann                   | Was                                                                  |
|------------------------|----------------------------------------------------------------------|
| zu Beginn jeder Stunde | Synchronisieren                                                      |
| sobald etwas läuft     | Commit mit Ticketnummer, dann Synchronisieren. Lieber oft als selten |
| Konflikt               | lösen wie im Git-Crashkurs, und mit dem anderen Team reden           |
| kaputt gemacht         | <code>Änderungen verwerfen</code> bringt den letzten Commit zurück   |

## KI-Notiz zu jedem Ticket

Ist ein Ticket erledigt, schreiben Sie in Ihre Ticketdatei unter das Ticket eine kurze Notiz. Die Ticketdatei gehört Ihrem Team, also gibt es dabei keine Konflikte.

`tickets/blasen.md`, unter dem Ticket

### ### KI-Notiz

- Weg und KI: Agent (GitHub Copilot)
- Anfragen: 4
- Was hat die KI gemacht? ...
- Was mussten wir korrigieren? ...

Die Notizen sind Material für KIZ.

## Für KIZ · Vibe Coding unter der Lupe

Wer der KI in Alltagssprache sagt, was er will, und den Code übernimmt, ohne ihn Zeile für Zeile zu lesen, macht Vibe Coding. Der Begriff stammt aus dem Jahr 2025. Im Projekt machen Sie das zum Teil: Sie schreiben den Code nicht selbst. Sie prüfen ihn aber über die Akzeptanzkriterien und über die grün und rot markierten Zeilen.

In KIZ werten Sie in Woche 2 Ihre KI-Notizen aus. Überlegen Sie sich bis dahin Antworten auf diese Fragen:

- Welche Ihrer Anfragen hat am meisten erreicht, und woran lag das?
- Wo lag die KI falsch? Woran haben Sie es gemerkt: am Test, an der Konsole oder beim Lesen des Codes?
- Was ist an einem Programm riskant, das niemand im Team Zeile für Zeile gelesen hat? Denken Sie an den Samstag, wenn am Stand etwas ausfällt.
- Wer beide Wege ausprobiert hat: Was konnte der Agent, was der Chat im Browser nicht konnte? Und umgekehrt?

### Die Erklärrunde

Jede Person erklärt in drei Minuten ihr Modul am Architektur-Schaubild aus `SCHNITTSTELLE.md`. Es geht um Abläufe, keine Syntax. Am Tag selbst muss jemand aus Ihrem Team einen Fehler finden können, ohne die KI zu fragen. Lesen Sie deshalb, was die KI geschrieben hat, spätestens dann, wenn das Ticket fertig ist.

### Geschafft, wenn ...

- ☐ Sie haben sich für Weg A oder B entschieden, und die KI antwortet Ihnen im Projektordner bzw. im Browser
- ☐ Bei Weg A: Sie kennen Ihren Kontostand unter [Chatnachrichten](#)
- ☐ Ticket 1 ist committet und synchronisiert, die anderen Teams sehen es
- ☐ Unter jedem erledigten Ticket steht eine KI-Notiz mit der Zahl der Anfragen
- ☐ Sie können am Schaubild zeigen, wo in Ihrer Datei das Neue passiert